Intern Group Project Briefs (Beginner Backend Version)

Project 1: Volunteer Management Platform

Project Overview

Organizations often work with volunteers during events and community activities. Managing volunteer information, events, and participation manually can become difficult.

Your task is to build a Volunteer Management Platform that helps organizations organize volunteers and manage activities efficiently.

Project Objective

The goal of this project is to apply backend development concepts learned during the internship, including:

- Creating a backend server.
- Building REST APIs.
- Organizing code into routes and controllers.
- Managing application data.
- Collaborating using Git and GitHub.

User Roles

Volunteer

A volunteer should be able to:

- Register as a volunteer.
- View their profile.
- View available events.
- Join an event.
- View their assigned activities.

Admin/Organization

An administrator should be able to:

- Create events.
- View volunteers.
- Assign volunteers to activities.
- Manage event information.
- Track participation.

Minimum Required Features (MVP)

Volunteer Management

- Create volunteer profiles.
- View all volunteers.
- View a specific volunteer.
- Update volunteer information.
- Delete volunteer information.

Event Management

- Create events.
- View available events.
- Register volunteers for events.
- View event participants.

Attendance Management

- Mark volunteer attendance.
- View participation records.

Technical Requirements

The application should include:

- Node.js and Express.js backend.
- REST API endpoints.
- Proper folder organization.
- Error handling.
- Data storage using arrays or JSON files.
- GitHub repository.

A database is not required for this version. The focus is understanding backend logic and API development.

Bonus Features

Teams may add:

- Search volunteers by skills.
- QR code attendance.

- Volunteer points system.
- Certificate generation.
- Notifications.

Project Deliverables

Each team should submit:

- GitHub repository.
- README documentation.
- List of API endpoints.
- Project structure explanation.
- Working demonstration.

Project 2: Mentor-Mentee Platform

Project Overview

Mentorship programs connect learners with experienced people who can guide them. Managing mentors, mentees, and mentorship activities manually can become challenging.

Your task is to create a platform that helps manage mentorship relationships.

Project Objective

This project will help you practice:

- Building multi-user applications.
- Managing relationships between users.
- Creating business logic.
- Designing more complex APIs.
- Working collaboratively as a development team.

User Roles

Mentor

A mentor should be able to:

- Create a mentor profile.
- Add skills and areas of expertise.
- View mentorship requests.
- Accept or reject requests.
- Provide feedback.

Mentee

A mentee should be able to:

- Create a profile.
- View available mentors.
- Request mentorship.
- Track mentorship progress.
- Provide feedback.

Admin

An administrator should be able to:

- View users.
- Manage mentor and mentee accounts.
- Monitor mentorship relationships.

Minimum Required Features (MVP)

User Management

- Create mentor profiles.
- Create mentee profiles.
- View profiles.
- Update profiles.

Mentorship Requests

- Send mentorship requests.
- Accept or reject requests.

- View active mentorship relationships.

Session Management

- Create mentorship sessions.
- View upcoming sessions.
- Add session feedback.

Technical Requirements

The application should include:

- Node.js and Express.js backend.
- REST API endpoints.
- Organized project structure.
- Data stored using arrays or JSON files.
- GitHub collaboration.

A database is not required for this version. The project should focus on backend architecture and application logic.

Bonus Features

Teams may add:

- Mentor search by skills.
- Rating system.
- Resource sharing.
- Notifications.
- Chat functionality.
- Progress tracking dashboard.

Final Presentation Requirements

Each team should present:

1. The problem they solved.
2. Their proposed solution.
3. Main features developed.
4. API demonstration.
5. Code organization.

6. Challenges encountered.
7. Lessons learned.
8. Future improvements.

Future Upgrade

After learning databases, these projects can be improved by adding:

- PostgreSQL or MongoDB integration.
- Real authentication.
- File uploads.
- Deployment.
- Advanced security features.